



Create Cluster

Understanding Kubernetes for Orchestration

Introduction

In Kubernetes, a **cluster** is a group of machines (nodes) working together to run containerized applications efficiently. Creating a Kubernetes cluster is the first step in managing and deploying applications using Kubernetes.

This guide will cover:

- **What is a Kubernetes cluster?**
 - **Why do we need it?**
 - **How to create a Kubernetes cluster using various methods**
 - **Practical steps with `kubeadm`, Minikube, and cloud providers**
 - **Common troubleshooting steps**
-

What is a Kubernetes Cluster?

A **Kubernetes cluster** consists of:

1. **Control Plane** – Manages the cluster, schedules workloads, and maintains the desired state.
2. **Worker Nodes** – Run the applications inside containers.
3. **Pods** – The smallest deployable units in Kubernetes.

Each cluster consists of a **master node (control plane)** and **one or more worker nodes**.

Why Do We Need a Kubernetes Cluster?

A Kubernetes cluster helps in:



- Managing containerized applications at scale
- Automating deployment, scaling, and operations
- Ensuring high availability and fault tolerance
- Efficient resource utilization

Without a cluster, managing multiple containers manually becomes difficult.

Methods to Create a Kubernetes Cluster

There are multiple ways to create a Kubernetes cluster, depending on your environment:

1. **Minikube** – Best for beginners and local development.
 2. **Kubeadm** – Ideal for setting up production-grade clusters.
 3. **Cloud Providers (AWS, GCP, Azure)** – Managed Kubernetes services like Amazon EKS, Google GKE, and Azure AKS.
-

Method 1: Creating a Cluster Using Minikube

Minikube is a lightweight tool to run a Kubernetes cluster locally.

Step 1: Install Minikube

1. **Install Minikube on Linux/macOS using curl:**

```
curl -LO  
https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64  
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

2. **Start Minikube:**

```
minikube start
```

3. **Verify Cluster is Running:**



```
kubectl get nodes
```

💡 *Minikube creates a single-node cluster for testing and development.*

Method 2: Creating a Cluster Using Kubeadm

kubeadm helps in setting up a **multi-node cluster** in a controlled way.

Step 1: Install Required Tools

Install Docker and Kubernetes dependencies:

```
sudo apt update  
sudo apt install -y docker.io  
sudo apt install -y kubeadm kubelet kubectl
```

Step 2: Initialize the Control Plane

On the master node, run:

```
sudo kubeadm init
```

Follow the instructions and set up **kubect**l:

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 3: Join Worker Nodes

On each worker node, run the **kubeadm join** command provided by the master node. Example:



```
sudo kubeadm join <master-ip>:6443 --token <your-token>
--discovery-token-ca-cert-hash sha256:<your-hash>
```

Verify all nodes are part of the cluster:

```
kubectl get nodes
```

💡 *This method is used for setting up clusters in real-world production environments.*

Method 3: Creating a Kubernetes Cluster on Cloud (AWS EKS Example)

Many organizations use managed Kubernetes services like **AWS Elastic Kubernetes Service (EKS)** for scalability and automation.

Step 1: Install AWS CLI & eksctl

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
brew install eksctl
```

Step 2: Create a Cluster

```
eksctl create cluster --name my-cluster --region ap-south-1
```

💡 *This creates a fully managed cluster in AWS.*



Common Issues and Troubleshooting

Problem	Solution
kubectl command not found	Ensure Kubernetes CLI is installed (kubectl version)
Nodes not joining	Check firewall settings and network connectivity
kubeadm init fails	Ensure docker and kubeadm versions are compatible
Pod not running	Check pod logs using kubectl logs <pod-name>

Know More (FAQs)

1. What is the difference between Minikube and Kubeadm?

- **Minikube** is for local testing and development.
- **Kubeadm** is used to create multi-node clusters in real environments.

2. Can I run Kubernetes on Windows?

Yes, you can use **Docker Desktop** with Kubernetes or **Minikube** on Windows.

3. How do I delete a Kubernetes cluster?

For Minikube:

```
minikube delete
```

For Kubeadm:

```
kubeadm reset
```



4. What is the best way to create a cluster for production?

- **Cloud-based Kubernetes (AWS EKS, Google GKE, Azure AKS)** is best for production.
- For on-premise clusters, use **kubeadm**.

5. How do I scale my cluster?

Use **kubectl scale** to scale deployments, or **add more worker nodes** to your cluster.

Conclusion

Creating a Kubernetes cluster is the first step towards **orchestrating containerized applications** efficiently. Whether using **Minikube, Kubeadm, or a Cloud Provider**, each method has its own use case.

👉 Start experimenting with Minikube on your local machine and explore how Kubernetes clusters work!